Лабораторная работа № 6

Библиотека Android Design Support Library

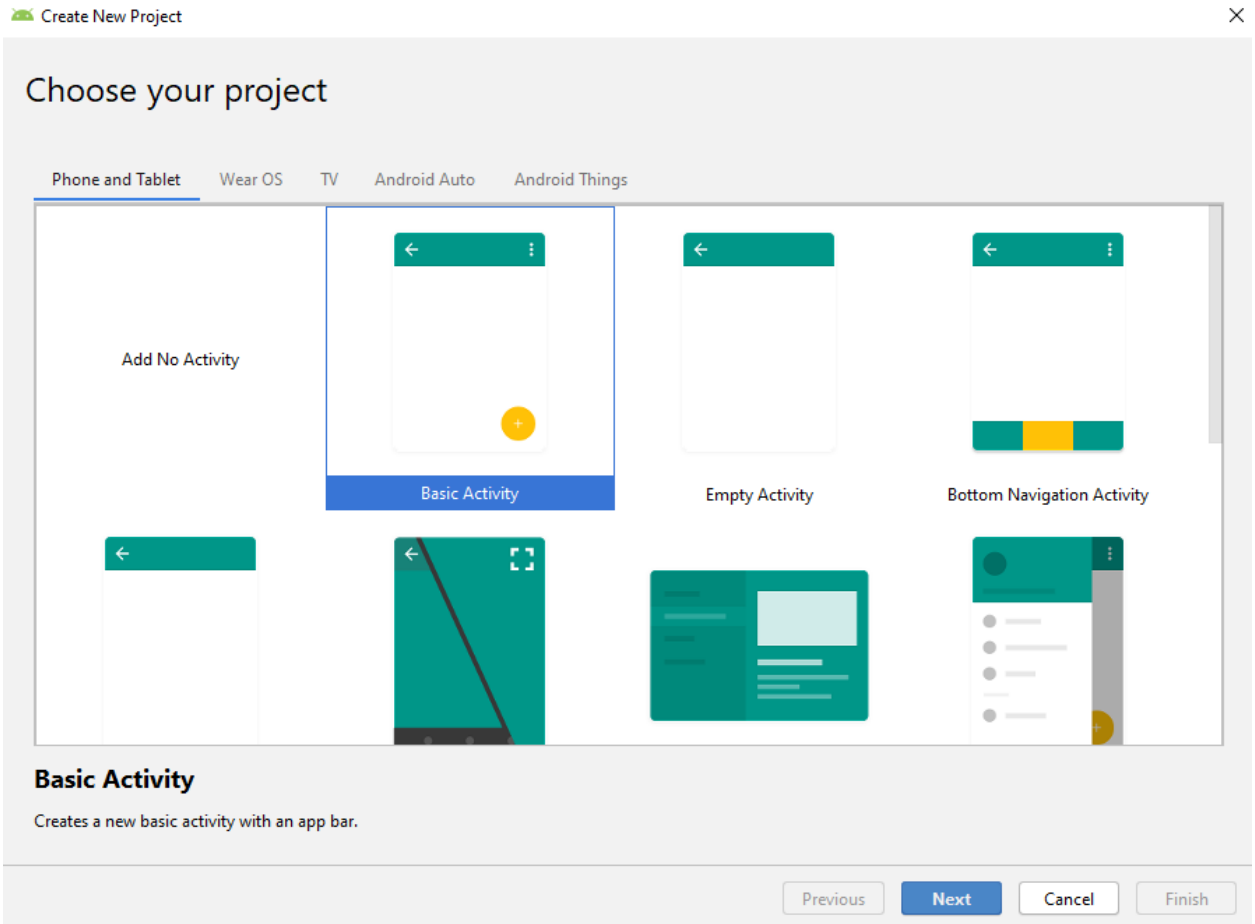
Цель работы: освоить некоторые возможности библиотеки **Android Design Support** для улучшения дизайна приложений.

Библиотека **Android Design Support** была представлена одновременно с версией Android Marshmallow, но её можно использовать на устройствах, начиная с версии Android 2.1. Библиотека была специально спроектирована под новый дизайн Material Design и позволяет использовать готовые компоненты в новом стиле.

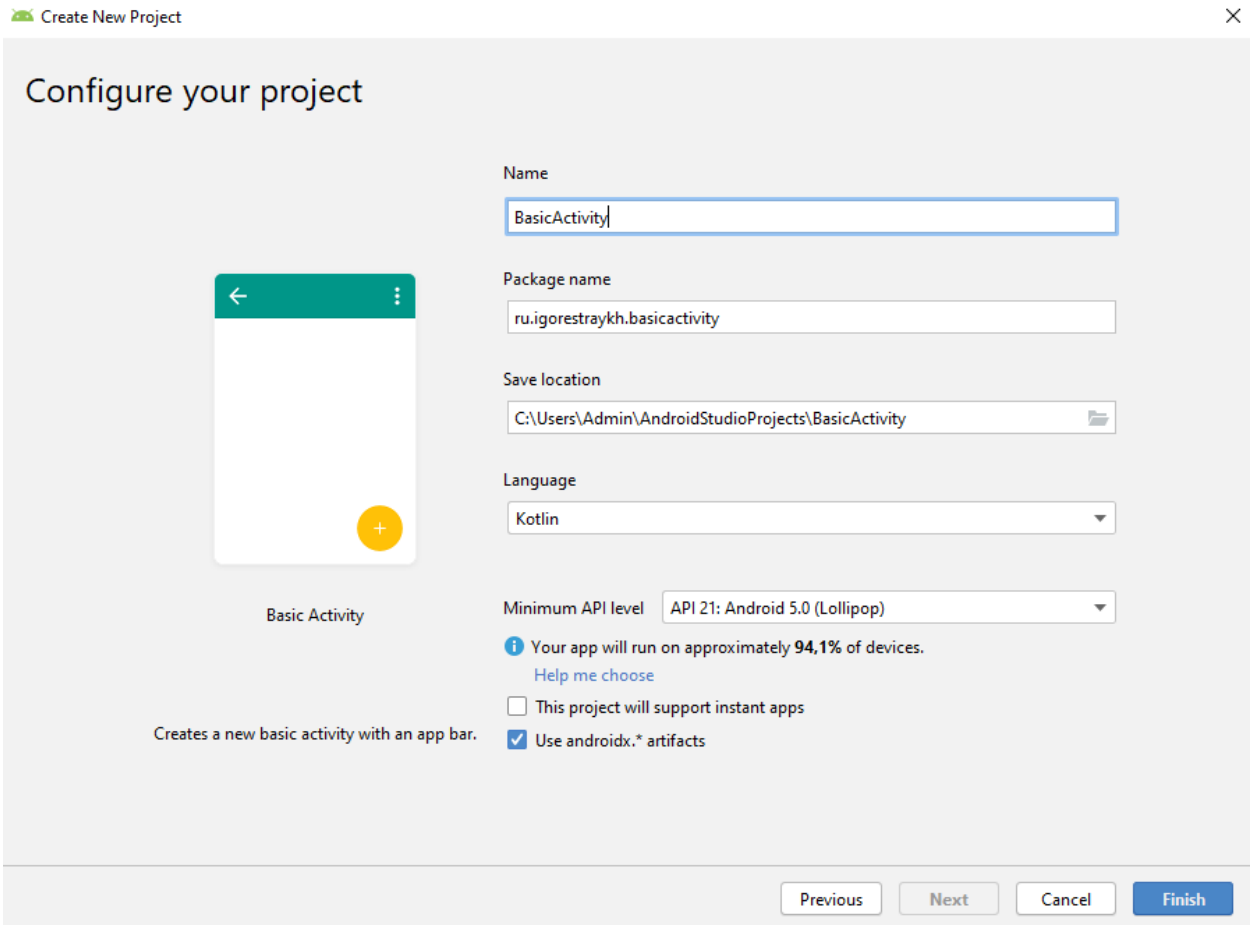
Material Design - это исчерпывающая кросс-платформенная концепция визуального оформления приложений, затрагивающая правила отображения анимации, общий стиль приложения и правила взаимодействия с управляющими элементами.

Мы можем использовать элементы библиотеки вручную, но для быстрого знакомства с некоторыми из них удобнее воспользоваться готовым шаблоном Android Studio.

Создайте проект при помощи шаблона **Basic Activity**.



В качестве языка выберите Kotlin.



При использовании нового шаблона будут задействованы четыре компонента библиотеки: **CoordinatorLayout**, **AppBarLayout**, **FloatingActionButton**, **Snackbar**.

Первые три компонента доступны через XML-разметку, а **Snackbar** вызывается программно.

Изучим разметку активности **activity_main.xml**.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
```

```
    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:theme="@style/AppTheme.AppBarOverlay">

        <androidx.appcompat.widget.Toolbar
            android:id="@+id/toolbar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
            android:background="?attr/colorPrimary"
            app:popupTheme="@style/AppTheme.PopupOverlay" />

    </com.google.android.material.appbar.AppBarLayout>

    <include layout="@layout/content_main" />

    <com.google.android.material.floatingactionbutton.FloatingActionButton
        android:id="@+id/fab"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom|end"
        android:layout_margin="@dimen/fab_margin"
        app:srcCompat="@android:drawable/ic_dialog_email" />

</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Корневым элементом разметки служит компонент **CoordinatorLayout**, который является наследником стандартного FrameLayout. Но, если обычный **FrameLayout** позволяет просто накладывать один компонент поверх другого, то **CoordinatorLayout** позволяет координировать определённые зависимости между дочерними компонентами.

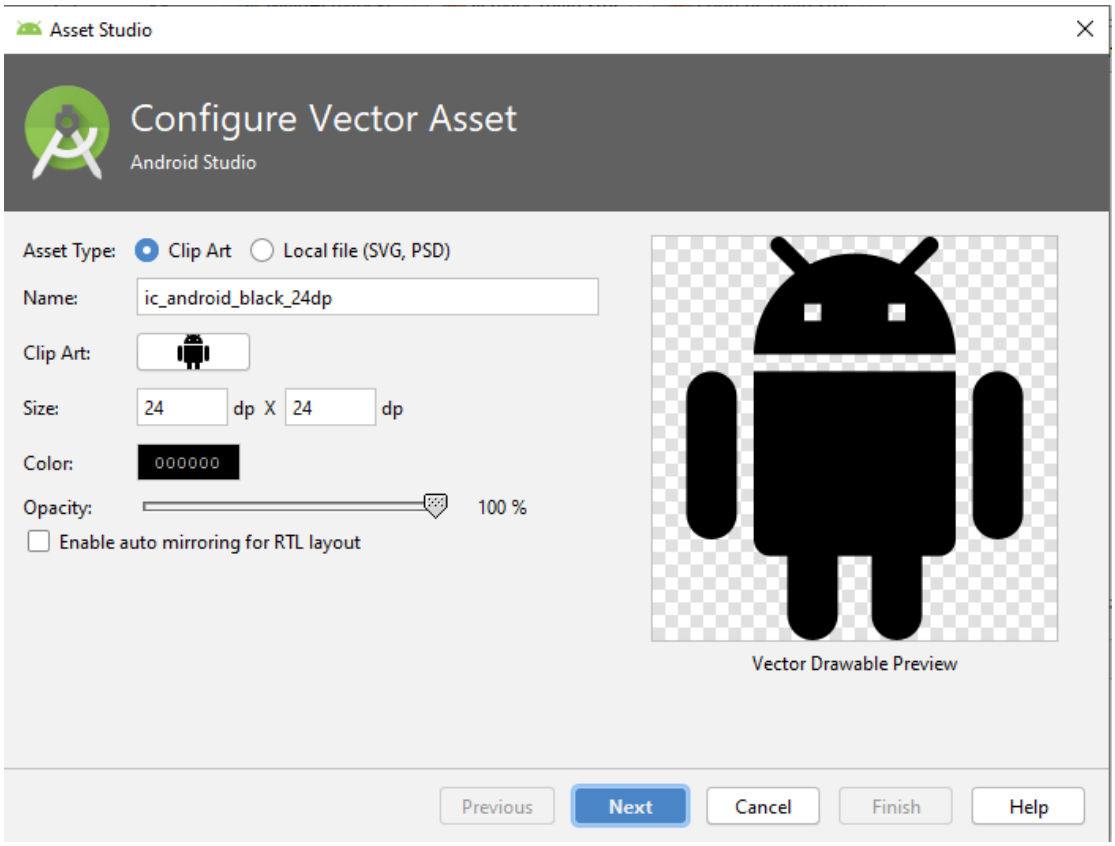
Далее идёт **AppBarLayout** с вложенным **Toolbar**. Связка компонентов образует внешний вид и поведение продвинутого заголовка экрана активности, который пришёл на смену **ActionBar** из Android 4.x, который в свою очередь заменил стандартный заголовок (Title) в Android 2.x.

Далее следует интересный приём, который вам может впоследствии пригодиться. В существующую разметку вставляется ещё одна разметка при помощи тега **include** и с указанием вставляемой разметки - **layout/content_main.xml**.

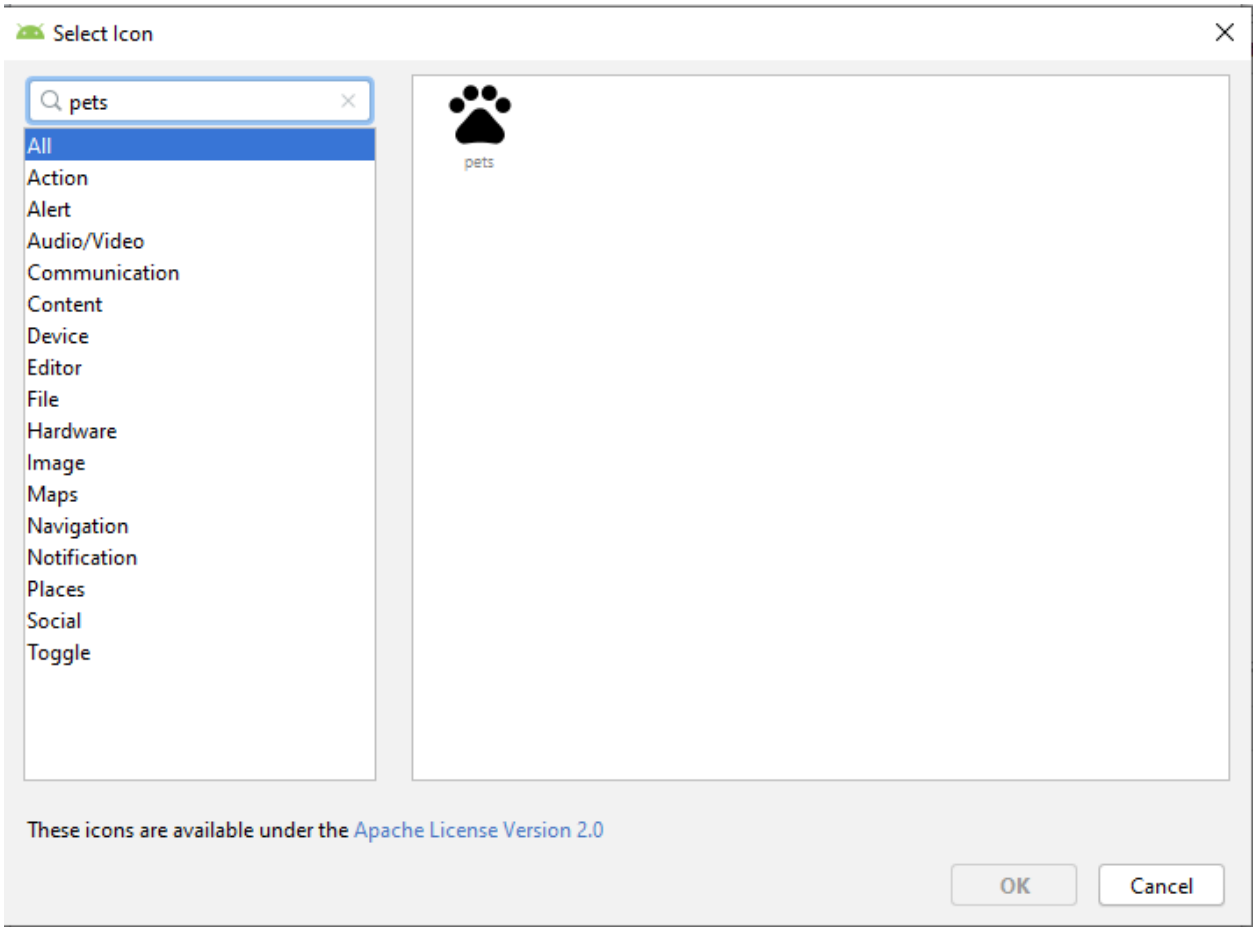
Завершает основную разметку красивая кнопка **FloatingActionButton**, которая на самом деле является продвинутым вариантом **ImageView**. Но в спецификации **Material Design** кнопке отводится большая роль, и она практически является визитной карточкой нового дизайна.

Обратите внимание, что кнопка размещена около нижнего правого угла экрана при помощи **layout_gravity**. В качестве значка используется изображение из системных ресурсов **@android:drawable/ic_dialog_email** в свойстве **srcCompat.**. Вы можете установить собственное изображение, подходящее по контексту. На данный момент Google рекомендует отказываться от растровых изображений и активно использовать векторные изображения.

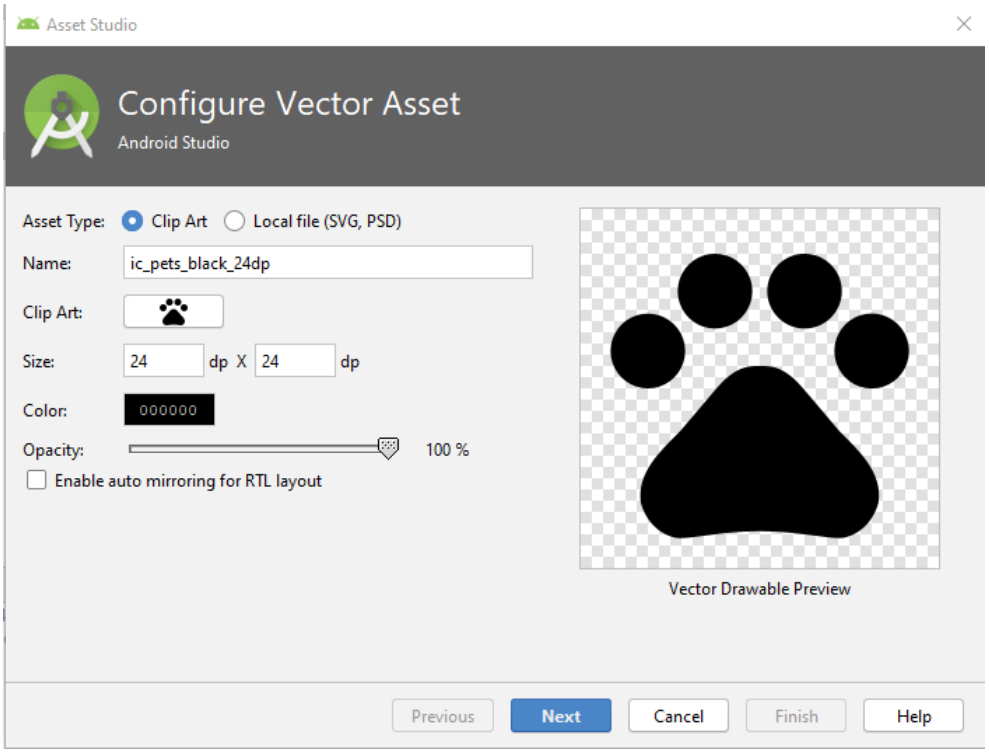
Заменим значок электронной почты, например, на изображение лапы животного. Щёлкаем правой кнопкой мыши по папке **app** и вызываем контекстное меню **New | Vector Asset**.



В диалоговом окне щёлкаем по значку **Clip Art**, чтобы открыть другое окно для выбора значка. В строке поиска набираем **pets** и находим нужный значок.

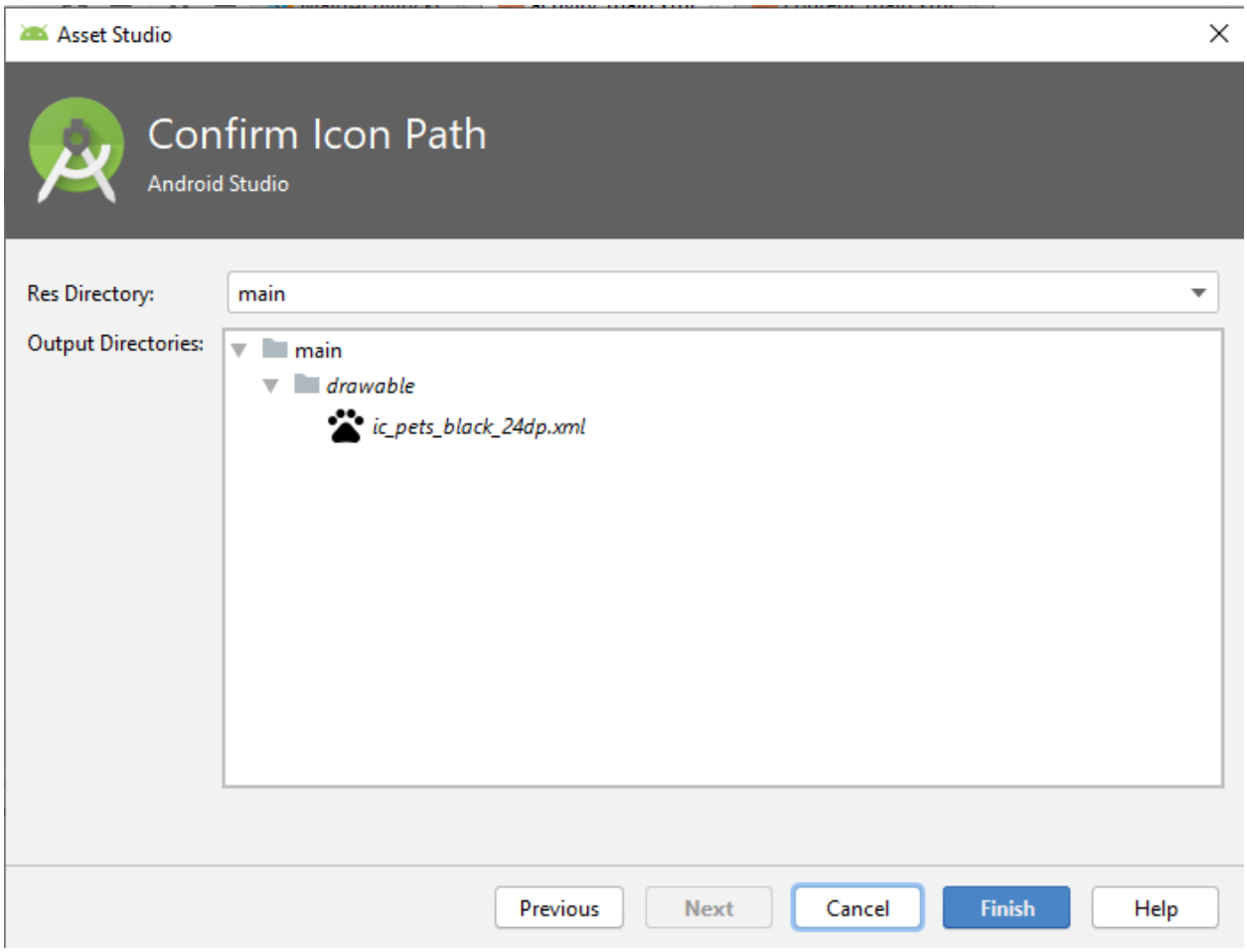


Выделяем его и нажимаем **OK**.

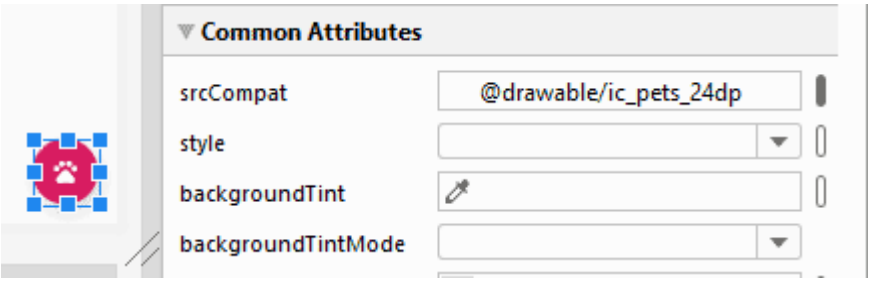


По умолчанию значок имеет чёрный цвет. Для нашего примера удобнее использовать белый цвет. В том же окне есть настройка цвета. Впрочем, цвет можно будет поменять и вручную позже.

Далее нажимаем кнопку **Next** и в следующем окне запоминаем название файла в ресурсах. Если вы всё-таки поменяли цвет, то логичнее будет поменять и название файла, убрав слово "black".



Теперь вы можете заменить значок для кнопки, выбрав соответствующий атрибут.



Базовый пример с Snackbar

Если раньше для всплывающих сообщений мы использовали **Toast**, то теперь можно использовать новый класс **Snackbar**, который имеет много общего с классом **Toast** и имеет практически тот же синтаксис.

Пример запуска сообщения:

```
// Kotlin
fab.setOnClickListener { view ->
    Snackbar.make(view, "Replace with your own action", Snackbar.LENGTH_LONG)
        .setAction("Action", null).show()
}

// Java
FloatingActionButton fab = findViewById(R.id.fab);
fab.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        Snackbar.make(view, "Replace with your own action", Snackbar.LENGTH_LONG)
                .setAction("Action", null).show();
    }
});
```

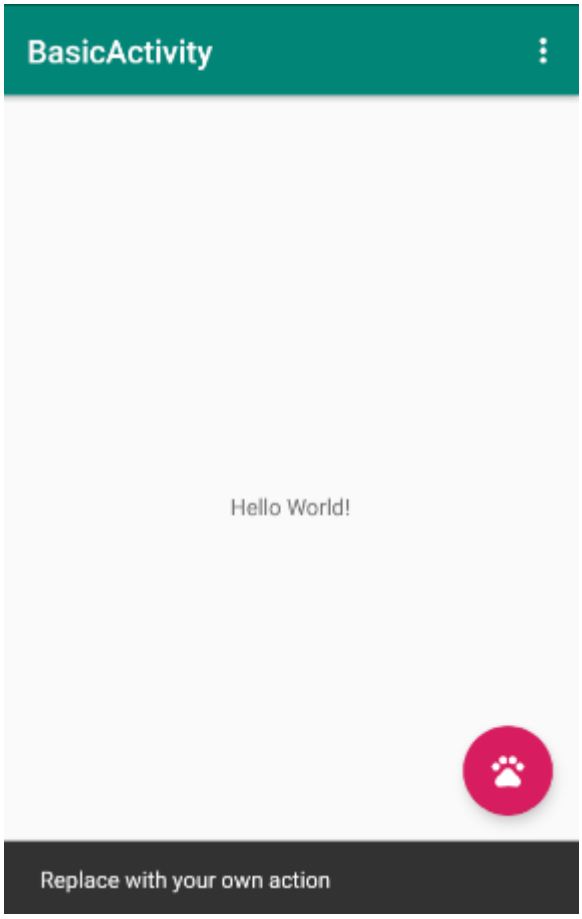
Как видите, код очень похож. Но есть и различия. Если **Toast** является частью активности и выводится поверх неё в нижней части по умолчанию, если не заданы другие параметры, то **Snackbar** выводится в "подвале" родительского элемента. В первом параметре указывается подходящий компонент, по которому система попытается найти родителя, обычно им является **CoordinatorLayout**. В некоторых примерах родитель указывается явно.

Код можно немного упростить, если убрать вызов метода **setAction()**, который сейчас не используется.

```
// Kotlin
```

```
Snackbar.make(view, "Replace with your own action", Snackbar.LENGTH_LONG)
    //.setAction("Action", null)
    .show()

// Java
Snackbar.make(view, "Replace with your own action", Snackbar.LENGTH_LONG).show();
```



Сообщение появляется на несколько секунд с указанным текстом и исчезает через несколько секунд. Для константы **LENGTH_SHORT** это будет 1.5 секунды, для **LENGTH_LONG** - 2.7 секунды. Практически аналогичное поведение у старого **Toast**.

Но есть и небольшие отличия. Например, когда выводится сообщение, его можно смахнуть с экрана слева направо, не дожидаясь, когда оно само исчезнет. Также можно задать свою продолжительность:

```
// Kotlin
```

```
val snackbar = Snackbar.make(view, "Replace with your own action", Snackbar.LENGTH_LONG)
snackbar.setAction("Action", null)
snackbar.duration = 8000
snackbar.show()

// Java
Snackbar snackbar = Snackbar.make(view, "Replace with your own action",
        Snackbar.LENGTH_LONG)
        .setAction("Action", null);
snackbar.setDuration(8000); // 8 секунд
snackbar.show();
```

В дальнейшем появилась новая константа **Snackbar.LENGTH_INDEFINITE** (неопределённая длительность). В этом случае компонент не исчезает самостоятельно.

Метод dismiss()

Метод **dismiss()** закрывает сообщение. Может пригодиться при использовании **LENGTH_INDEFINITE**, когда сообщение будет висеть на экране вечно. Добавим кнопку для закрытия сообщения и напишем код (Java).

```
private Snackbar mSnackbar;

@Override
protected void onCreate(Bundle savedInstanceState) {
   super.onCreate(savedInstanceState);
   setContentView(R.layout.activity_main);
   Toolbar toolbar = findViewById(R.id.toolbar);
   setSupportActionBar(toolbar);

   FloatingActionButton fab = findViewById(R.id.fab);
   fab.setOnClickListener(new View.OnClickListener() {
```

```
            @Override
            public void onClick(View view) {
                mSnackbar = Snackbar.make(view, "Пора изучать Java!",
                        Snackbar.LENGTH_INDEFINITE)
                        .setAction("Action", null);
                mSnackbar.show();
            }
        });

        Button dismissButton = findViewById(R.id.button_dismiss);
        dismissButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                mSnackbar.dismiss();
            }
        });
    }
```

Чтобы **Snackbar** можно было убирать движением пальца, компонент следует поместить в **CoordinatorLayout**. В стандартных разметках (**LinearLayout** и ему подобных) операция не сработает.

Snackbar.События

Отслеживать события появления и исчезновения **Snackbar** можно с помощью методов обратного вызова через **addCallback()**.

Параметр **event** у метода **onDismissed()** позволяет узнать конкретное событие, повлекшее исчезновение, при помощи констант **DISMISS_EVENT_SWIPE, DISMISS_EVENT_ACTION, DISMISS_EVENT_TIMEOUT, DISMISS_EVENT_MANUAL, DISMISS_EVENT_CONSECUTIVE**.

```
    mSnackbar = Snackbar.make(view, "Пора изучать Java!",
            Snackbar.LENGTH_LONG)
```

```
            .setAction("Action", null);

        mSnackbar.show();

        mSnackbar.addCallback(new Snackbar.Callback() {

            @Override
            public void onDismissed(Snackbar snackbar, int event) {
                if (event == Snackbar.Callback.DISMISS_EVENT_TIMEOUT) {
                    Log.i("SnackBar", "Закрыт по истечении таймаута");
                }
                if(event == Snackbar.Callback.DISMISS_EVENT_SWIPE){
                    Log.i("SnackBar", "Swipe");
                }
            }

            @Override
            public void onShown(Snackbar snackbar) {
                Log.i("SnackBar", "onShown");
            }
        });
```

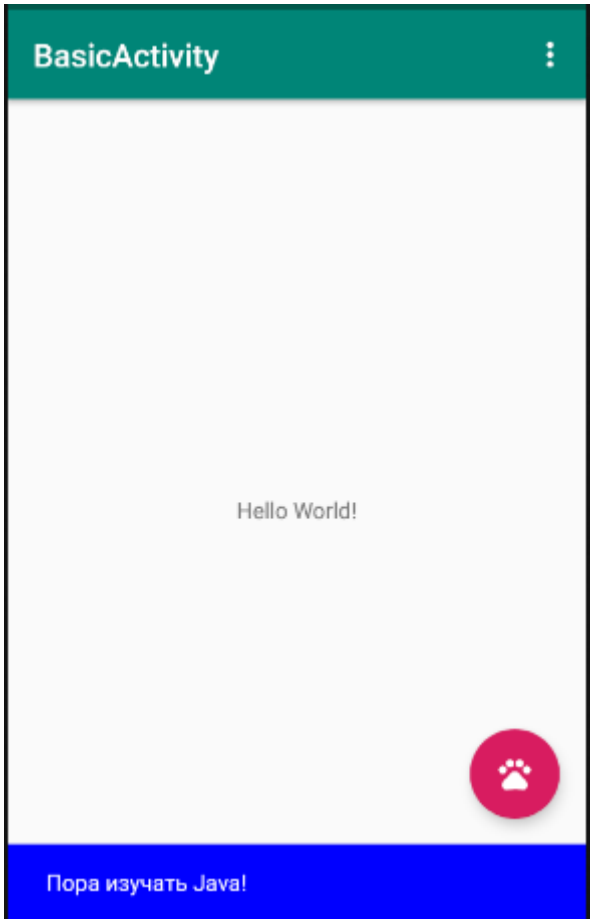
Меняем цвет текста и фона

Класс **Snackbar** является закрытым ящиком, и мы не знаем, как он устроен. Но у него есть метод **getView()**, который возвращает некий компонент **View**. Получив его, мы можем поменять у него фоновый цвет.

```
        fab.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                mSnackbar = Snackbar.make(view, "Пора изучать Java!",
```

```
                    Snackbar.LENGTH_INDEFINITE)
                .setAction("Action", null);

            View snackbarView = mSnackbar.getView();
            snackbarView.setBackgroundColor(Color.BLUE);
            mSnackbar.show();
        }
    });
```

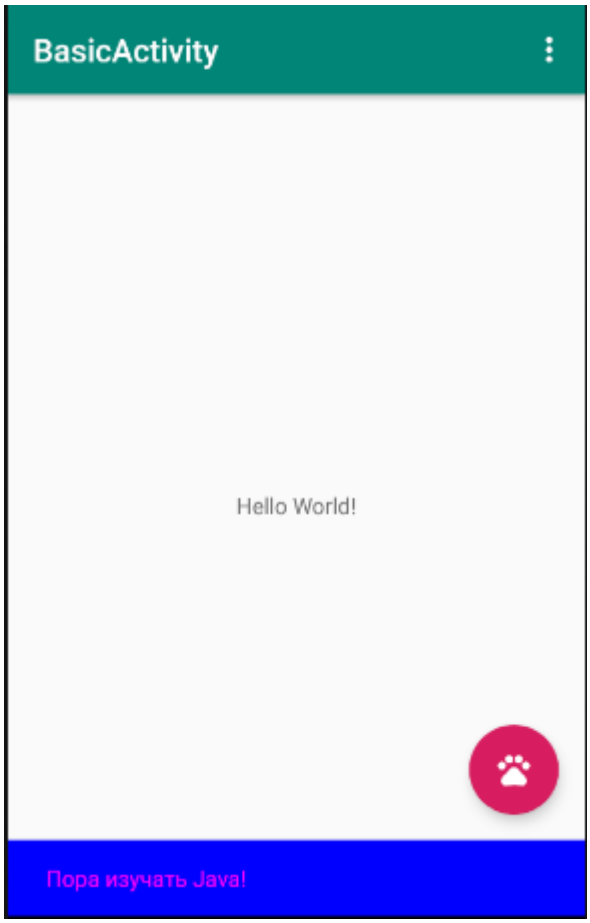


Если вам захочется поменять также и цвет текста в сообщении, то нужно получить доступ к **TextView**:

```
    mSnackbar = Snackbar.make(view, "Пора изучать Java!", Snackbar.LENGTH_INDEFINITE)
            .setAction("Action", null);

    View snackbarView = mSnackbar.getView();
```

```
snackbarView.setBackgroundColor(Color.BLUE);
//TextView snackTextView = snackbarView.findViewById(android.support.design.R.id.snackbar_text); // старый вариант
TextView snackTextView = snackbarView.findViewById(com.google.android.material.R.id.snackbar_text);
snackTextView.setTextColor(Color.MAGENTA);
mSnackbar.show();
```



Добавляем кнопку действия

Ради вывода сообщения в подвале экрана не стоило создавать новый класс, **Toast** вполне справлялся с этой задачей. Теперь рассмотрим новую возможность. На панели **Snackbar** можно разместить кнопку действия с помощью метода **setAction()**. В методе нужно указать текст для кнопки и обработчик для щелчка.

```
@Override
protected void onCreate(Bundle savedInstanceState) {
```

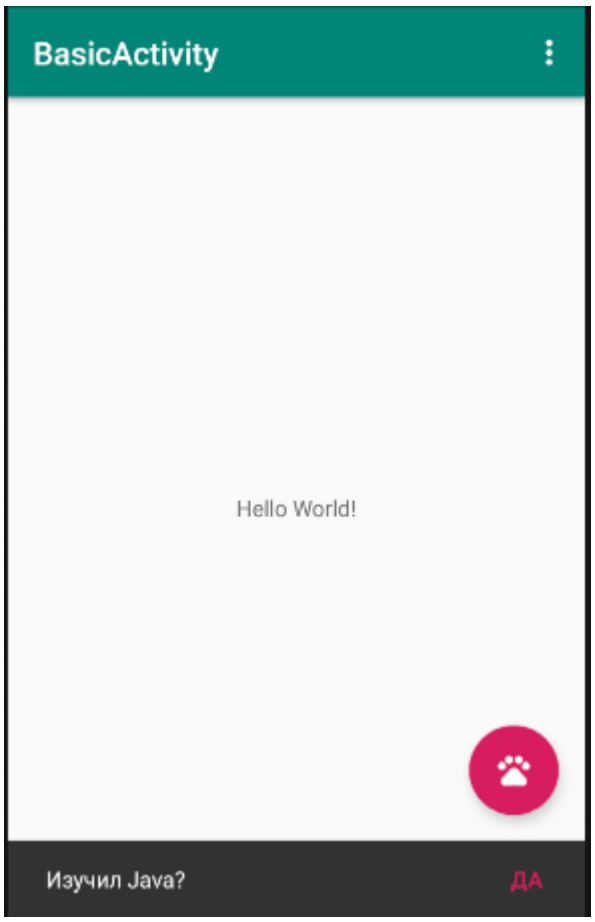
```
        super.onCreate(savedInstanceState);


        ...


        FloatingActionButton fab = (FloatingActionButton) findViewById(R.id.fab);
        fab.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                mSnackbar = Snackbar.make(view, "Изучил Java?",
                                Snackbar.LENGTH_INDEFINITE)
                        .setAction("Да", snackbarOnClickListener);
                mSnackbar.show();
            }
        });
    }


    View.OnClickListener snackbarOnClickListener = new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            Toast.makeText(getApplicationContext(), "Молодец!",
                    Toast.LENGTH_LONG).show();
        }
    };
```

Как правило, при нажатии на кнопку действия панель исчезает. Если этого не происходит, всегда можно добавить дополнительную строчку с вызовом метода **dismiss()**.



С помощью метода **setActionTextColor()** можно поменять цвет текста у кнопки действия.

```
Snackbar
    .make(parent, "Изучил Java?", Snackbar.LENGTH_LONG)
    .setAction("Да", snackOnClickListener)
    .setActionTextColor(Color.MAGENTA) // цвет текста у кнопки действия
    .show();
```

Если цвет не задавать явно, то будет использован стиль **colorAccent**. Для изменения цвета обычного текста в **Snackbar** можно установить стиль **android:textColor** (он подействует на все компоненты на экране).

```
<style name="SnackbarTheme" parent="Theme.AppCompat.Light">
  <item name="colorAccent">@color/orange</item>
  <item name="android:textColor">@color/blue</item>
</style>
```

```
...
<activity
    ...
    android:theme="@style/AppTheme">
</activity>
```

Приподнимаем компонент

Вы, возможно, замечали, что FAB приподнимается при появлении **SnackBar**. Такую же возможность можно добавить для компонента, который находится в нижней части экрана. Чтобы **SnackBar** не загораживал обзор, нужно для компонента добавить новый атрибут **app:layout_dodgeInsetEdges="bottom"**. Это сработает, если компонент является дочерним элементом разметки **CoordinatorLayout**. Для других разметок и для других степеней вложенности приём не сработает.

Настраиваем SnackBar

Стандартный компонент имеет ширину экрана в телефонах. Но мы можем изменить его поведение. Создадим уменьшенный прямоугольник с закруглёнными углами. Для начала создадим ресурс в папке **res/layout** под именем **bg_snackbar.xml**, который послужит подложкой.

```
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="#323232" />
    <corners android:radius="4dp" />
</shape>
```

Добавим новый класс **SnackbarHelper**.

```
package ru.igorestraykh.basicactivity;


import android.content.Context;
import android.support.design.widget.Snackbar;
import android.support.v4.view.ViewCompat;
import android.view.ViewGroup;
```

```
class SnackbarHelper {

    static void configSnackbar(Context context, Snackbar snackbar) {
        addMargins(snackbar);
        setRoundBordersBg(context, snackbar);
        ViewCompat.setElevation(snackbar.getView(), 6f);
    }

    private static void addMargins(Snackbar snack) {
        ViewGroup.MarginLayoutParams params = (ViewGroup.MarginLayoutParams) snack.getView().getLayoutParams();
        params.setMargins(12, 12, 12, 12);
        snack.getView().setLayoutParams(params);
    }

    private static void setRoundBordersBg(Context context, Snackbar snackbar) {
        // API 21

snackbar.getView().setBackground(context.getDrawable(R.drawable.bg_snackbar));
    }
}
```

Заменим вызов **SnackBar**.

```
Snackbar snackbar = Snackbar.make(view,
        "Пора изучать Java!", Toast.LENGTH_SHORT);
SnackbarHelper.configSnackbar(MainActivity.this, snackbar);
//snackbar.config(this) if you're using Kotlin
snackbar.show();
```

Задание на лабораторную работу

1. Создайте приложения на Java и Kotlin, использующие возможности библиотеки **Android Design Support**, рассмотренные в методических указаниях.

2. Включите в отчет цель работы, коды программ, скриншоты с результатами и вывод.